

LLM from scratch notes

Ashirbad Sahu
29th July 2026

Table of Contents

LLM from scratch using numpy and cupy	4
Elements of an LLM	4
Tokenization	5
BBPE (Bytelevel Byte Pair Encoding)	5
How BBPE works	5
Example	5
Positional Encoding	8
Why do we need positional encoding?	8
Example:	8
The Problem:	8
The Solution:	8
Math of Positional Encoding in the Project	8
Capturing Relative Distance via Linear Combination	8
Why this matters:	9
Feed Forward Networks (FFNs)	10
What is a Feed Forward Network?	10
Activation Functions	10
Gaussian Error Linear Unit (GELU)	10
Attention Mechanism	11
Mathematical Definition	11
Step-by-Step Breakdown:	11
Intuitive Example	11
Multi-Head Attention	12
Mathematical Definition	12
Step-by-Step Breakdown	12
Intuitive Example	12
Causal Masking (Lower Triangular Mask)	13
Mathematical Definition	13
Step-by-Step Breakdown	13
Intuitive Example	13
Transformer Block: Layer Normalization & Residual Connections	14
1. Layer Normalization (LayerNorm)	14
What is Convergence?	14
Why Do We Need LayerNorm?	14
Mathematical Definition	14
Key Difference: LayerNorm vs. BatchNorm	14
2. Residual Connections (Skip Connections)	14
Why Do We Need Residual Connections?	14
Standard Transformer Block Architecture (Pre-LN)	15
Mathematical Proof: Solving Vanishing Gradients	15
Transformer Blocks: Encoder vs. Decoder	16
Transformer Encoder Block	16
Forward Pass Mathematics	16
Transformer Decoder Block	16
Forward Pass Mathematics	16
3. Architectural Comparison	16

Complete Model Architecture and Training Loop	18
Mathematical pipeline:	18
Cross-Entropy Loss	18
Adam Optimizer	19
Mathematical Formulation	19
Data Pipeline and Training at Scale	20
Gradient Accumulation	20
Algorithm	20
Learning Rate Scheduling	20
Warmup Phase	20
Cosine Decay Phase	20
Symbol Definitions	20
Evaluation Metrics	21
Perplexity (PPL)	21

LLM from scratch using numpy and cupy

This is the documentation and notes for my project to build a large language model (LLM) from scratch using numpy and cupy. The goal of this project is to understand the inner workings of LLMs by implementing one from the ground up, without relying on high-level libraries like PyTorch or TensorFlow.

Elements of an LLM

- Tokenization
- Embeddings
- Attention Mechanism
- Transformer Architecture
- Training Loop
- Optimization
- Evaluation Metrics

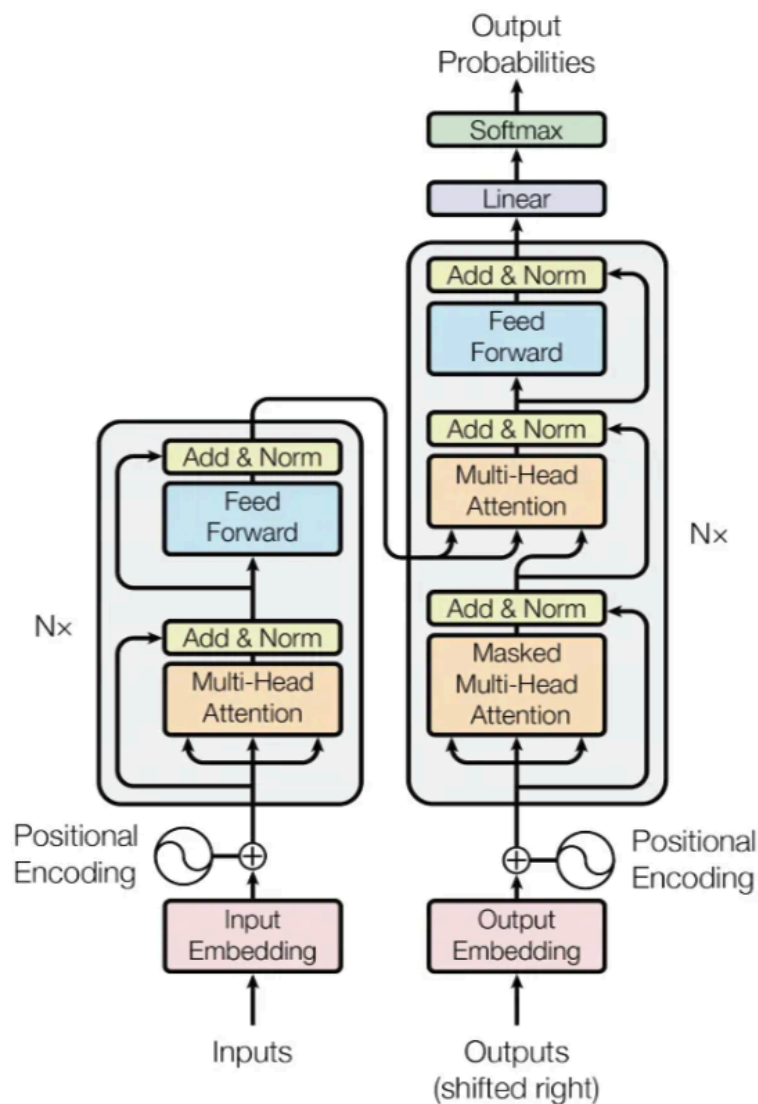


Figure 1: Original transformer architecture

Tokenization

Converting raw text into numerical representations called tokens, which can be processed by the model. Common tokenization techniques include:

- Word-level tokenization
- Subword tokenization (e.g., Byte Pair Encoding)
- Character-level tokenization

I have implemented **Bytelevel Byte Pair Encoding** (BBPE) for my LLM, which allows for efficient handling of out-of-vocabulary words and better representation of rare words.

BBPE (Bytelevel Byte Pair Encoding)

BBPE starts with a vocabulary containing all possible byte values.

Since a byte can represent values from 0 to 255, the initial vocabulary size is:

$$|V_0| = 256$$

This guarantees that any UTF-8 text can always be represented without needing an unknown token ().

Let: N = corpus length M = number of merge operations A naive implementation requires: $O(MN)$ time complexity, because pair frequencies may need recomputation after every merge. However, with efficient data structures and caching, the complexity can be reduced.

How BBPE works

- Start by cleaning and preprocessing the text data. Normalize whitespace, remove unwanted characters if needed, and make sure everything is UTF-8 encoded so the tokenizer can work consistently across different languages and symbols.
- Split the text into bytes instead of characters or words. Since BBPE operates directly on raw bytes, every piece of text can be represented without needing an unknown token.
- Add special markers or boundaries where required (like `</w>`). Some implementations keep track of word boundaries or whitespace so merges do not accidentally combine unrelated parts of the text.
- Flatten the entire corpus into one long sequence of byte-level tokens. At this stage, every byte is treated as an individual token in the vocabulary.
- Count all adjacent token pairs across the corpus and track how often each pair appears.
- Find the most frequent pair of consecutive tokens. This pair is assumed to represent a commonly occurring pattern in the data.
- Merge that pair into a new single token and add it to the vocabulary.
- Replace every occurrence of that pair in the corpus with the newly created merged token.
- Recalculate pair frequencies because the merges change the surrounding token combinations.
- Repeat the process: count pairs, merge the most frequent one, and update the corpus after each merge.
- Continue merging until the desired vocabulary size is reached or no useful merges remain.
- Save the final vocabulary and merge rules. These rules are later used to tokenize new text by replaying the same merge operations in the same order.

Example

Suppose our training corpus contains the sentence:

Players played before playing

BBPE first converts the text into smaller units (bytes or characters for simplicity in this example).

```
[P] [l] [a] [y] [e] [r] [s]
[p] [l] [a] [y] [e] [d]
[b] [e] [f] [o] [r] [e]
[p] [l] [a] [y] [i] [n] [g]
```

Step 1: Count adjacent pairs

We count how often every adjacent pair appears across the corpus.

Some frequent pairs are:

```
(p, l) -> 2
(l, a) -> 2
(a, y) -> 2
(y, e) -> 2
```

Assume the most frequent pair selected is:

```
(p, l) -> pl
```

Step 2: Merge the pair

Replace every occurrence of (p, l) with the new token pl.

```
[P] [l] [a] [y] [e] [r] [s]
[pl] [a] [y] [e] [d]
[b] [e] [f] [o] [r] [e]
[pl] [a] [y] [i] [n] [g]
```

Now we recalculate pair frequencies.

Some updated pairs become:

```
(pl, a) -> 2
(a, y) -> 2
```

Suppose we now merge:

```
(pl, a) -> pla
```

Step 3: Merge again

After applying the merge:

```
[P] [l] [a] [y] [e] [r] [s]
[pla] [y] [e] [d]
[b] [e] [f] [o] [r] [e]
[pla] [y] [i] [n] [g]
```

The pair (pla, y) also appears frequently, so we merge it next:

```
(pla, y) -> play
```

After the merge:

```
[P] [l] [a] [y] [e] [r] [s]
[play] [e] [d]
[b] [e] [f] [o] [r] [e]
[play] [i] [n] [g]
```

Learned Subword Tokens

Over time, BBPE learns commonly occurring subwords such as:

pl
pla
play
ing
ed

So instead of storing every word independently, the tokenizer can represent words using reusable subword units.

For example:

played -> [play] [ed]
playing -> [play] [ing]

This is what makes BBPE efficient: it learns reusable patterns that appear across many different words.

Positional Encoding

Why do we need positional encoding?

Transformers use self attention, which is permutation-invariant (order insensitive). Without positional information, the model cannot distinguish between sequences with the same tokens in different orders.

Example:

- **Sentence A:** “Only **he** wants to eat biryani.” (No one else wants to eat it)
- **Sentence B:** “**He** wants to eat only biryani.” (He wants to eat nothing else)

The Problem:

Both sentences contain the exact same tokens: {"only", "he", "wants", "to", "eat", "biryani"}.

Without positional encoding, the Transformer cannot determine which word “only” is modifying. It generates identical mathematical representations for both sentences, completely failing to capture the difference in meaning.

The Solution:

Positional encoding adds a unique structural “coordinate” to each word vector before it enters the attention mechanism:

- In Sentence A, “only” is tagged at **Position 1** (attending closely to “he”).
- In Sentence B, “only” is tagged at **Position 5** (attending closely to “biryani”).

This extra mathematical information breaks the order-insensitivity, allowing the model to correctly interpret the syntax and context.

Math of Positional Encoding in the Project

In this project, a deterministic **sinusoidal** positional embedding is used to inject sequence order into the model.

- **For even dimension indices ($2i$):**

$$\text{PE}_{\text{pos}, 2i} = \sin\left(\frac{\text{pos}}{10000^{\frac{2i}{d_e}}}\right)$$

- **For odd dimension indices ($2i + 1$):**

$$\text{PE}_{\text{pos}, 2i+1} = \cos\left(\frac{\text{pos}}{10000^{\frac{2i}{d_e}}}\right)$$

Where:

- pos : The absolute token position in the sequence (0 to $T - 1$).
- i : The frequency dimension index (0 to $\frac{d_e}{2} - 1$).
- d_e : The total embedding dimension.

Capturing Relative Distance via Linear Combination

The core genius of the sine-cosine formulation is that it allows the model to easily learn *relative* distances. For any fixed token offset k , the positional encoding at position $\text{pos} + k$ can be represented as a **linear transformation (rotation)** of the encoding at position pos .

For a specific frequency component $\omega_i = \frac{1}{10000 \frac{2i}{d_e}}$, we can pair the even (sine) and odd (cosine) dimensions into a 2D vector space. Using the trigonometric angle addition identities:

$$\sin(\omega_i(\text{pos} + k)) = \sin(\omega_i \text{ pos}) \cos(\omega_i k) + \cos(\omega_i \text{ pos}) \sin(\omega_i k)$$

$$\cos(\omega_i(\text{pos} + k)) = \cos(\omega_i \text{ pos}) \cos(\omega_i k) - \sin(\omega_i \text{ pos}) \sin(\omega_i k)$$

This can be elegantly refactored into a matrix multiplication (a linear combination):

$$\begin{pmatrix} \text{PE}_{\text{pos}+k, 2i} \\ \text{PE}_{\text{pos}+k, 2i+1} \end{pmatrix} = \begin{pmatrix} \cos(\omega_i k) & \sin(\omega_i k) \\ -\sin(\omega_i k) & \cos(\omega_i k) \end{pmatrix} \begin{pmatrix} \text{PE}_{\text{pos}, 2i} \\ \text{PE}_{\text{pos}, 2i+1} \end{pmatrix}$$

Why this matters:

1. **Position-Independent Transformation:** The 2×2 rotation matrix depends **only** on the relative offset k , not on the absolute position pos .
2. **Attention Mechanism Compatibility:** When the self-attention layer calculates dot-products ($Q^T K$) between two tokens separated by distance k , the resulting attention score is a direct function of this relative rotation matrix. This enables the model to easily generalize to tracking how far apart words are from each other regardless of where they appear in a sentence.

Feed Forward Networks (FFNs)

What is a Feed Forward Network?

An FFN (Feed-Forward Neural Network) is a standard layer of artificial neurons where data moves in one direction only (forward, from input to output through hidden layers).

- Mathematically:

$$\text{FFN}(x) = \text{Activation}(xW_1 + b_1)W_2 + b_2$$

Where:

- x : Input vector
- W_1 : First weight matrix, projects x into a much higher-dimensional space
- b_1 : First bias vector
- $\text{Activation}()$: Non-linear activation function
- W_2 : Second weight matrix, compresses expanded features back to model's standard size
- b_2 : Second bias vector
- $\text{FFN}(x)$: Output vector

Activation Functions

- Activation functions are applied to every number in the vector. They decide which values pass through and which get muted or zeroed out, allowing the network to learn complex, non-linear relationships rather than simple linear algebra.
- Common activation functions include GELU, SwiGLU, ReLU, Sigmoid, and Tanh.
- In this project, **GELU (Gaussian Error Linear Unit)** is used.

Gaussian Error Linear Unit (GELU)

GELU scales inputs by their probability under a Gaussian distribution, providing a smooth, non-linear transformation that allows small negative gradients to flow.

- Exact Definition:

$$\text{GELU}(x) = x \cdot \Phi(x) = x \cdot \frac{1}{2} \left[1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right) \right]$$

Where:

- $\Phi(x)$: Cumulative Distribution Function (CDF) of standard normal distribution
- $\text{erf}(x)$: Error function calculating area under a bell curve
- Fast Approximation:

For computational efficiency, frameworks commonly approximate GELU as:

$$\text{GELU}(x) \approx 0.5x \cdot \left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3)\right) \right)$$

Attention Mechanism

Attention allows the model to focus on relevant parts of the input sequence when processing each token by assigning different weights to different tokens.

Mathematical Definition

Given:

- Query matrix $Q \in \mathbb{R}^{T \times d_k}$ (what we are looking for)
- Key matrix $K \in \mathbb{R}^{T \times d_k}$ (what we match against)
- Value matrix $V \in \mathbb{R}^{T \times d_v}$ (what we retrieve), where d_k and d_v are the key and value dimensions respectively

Attention is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Step-by-Step Breakdown:

1. Compute similarity scores:

$$\text{scores} = QK^T \in \mathbb{R}^{T \times T}$$

Measures how much each token in the sequence relates to every other token.

2. Scale scores:

$$\text{scaled_scores} = \frac{QK^T}{\sqrt{d_k}}$$

Scales down the dot products by $\sqrt{d_k}$ to prevent extremely large values, which cause vanishing gradients in the softmax function.

3. Apply Softmax:

$$\text{weights} = \text{softmax}(\text{scaled_scores})$$

Normalizes the scores across each row into probabilities between 0 and 1 that sum to 1.

4. Weight the Values:

$$\text{output} = \text{weights} \cdot V \in \mathbb{R}^{T \times d_v}$$

Computes a weighted sum of the Value vectors using the softmax attention probabilities to form the final representation.

Intuitive Example

Consider a simple three-word sentence: “**The cat sat**”

For these three words, the model projects Query (Q), Key (K), and Value (V) matrices:

$$Q = [q_{\text{The}}, q_{\text{cat}}, q_{\text{sat}}]$$

$$K = [k_{\text{The}}, k_{\text{cat}}, k_{\text{sat}}]$$

$$V = [v_{\text{The}}, v_{\text{cat}}, v_{\text{sat}}]$$

When the model is specifically **processing the word “sat”**, it takes the query vector q_{sat} and compares it against all key vectors (k) using dot products to measure relevance:

- $q_{\text{sat}} \cdot k_{\text{cat}}$ is **high**: “Sat” (an action) strongly relates to “cat” (the noun performing the action).
- $q_{\text{sat}} \cdot k_{\text{The}}$ is **medium**: “The” provides minor grammatical context.
- $q_{\text{sat}} \cdot k_{\text{sat}}$ is **low**: “Sat” doesn’t need to look at itself as much to gain extra meaning here.

1. Softmax converts these similarity scores into attention weights (percentages):

$$\text{softmax}([\text{medium}, \text{high}, \text{low}]) \rightarrow [0.2, 0.7, 0.1]$$

2. Constructing the final output: The model uses these weights to combine the information vectors (v):

$$\text{Output} = 0.2v_{\text{The}} + 0.7v_{\text{cat}} + 0.1v_{\text{sat}}$$

Result: The final representation for “sat” now carries **mostly information about the “cat”**, giving the model the context that **a cat is the thing sitting**.

Multi-Head Attention

Instead of performing attention once across the entire vector dimension, **Multi-Head Attention** splits the Query, Key, and Value representations into h smaller “heads”. This allows the model to jointly attend to information from different representation subspaces at different positions.

Mathematical Definition

Given number of heads h , and projection matrices $W_i^Q, W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$, and $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h)W^O$$

Where each individual head i is calculated as:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Step-by-Step Breakdown

1. **Linear Projections:** Project input matrices Q , K , and V into h lower-dimensional subspaces using weight matrices W_i^Q , W_i^K , and W_i^V .
2. **Parallel Attention:** Compute scaled dot-product attention for each head independently in parallel.
3. **Concatenation:** Concatenate the outputs from all h heads along the feature dimension.
4. **Final Output Projection:** Multiply the concatenated result by the output projection matrix W^O to restore the vector back to d_{model} .

Intuitive Example

Consider the sentence: **“The animal didn’t cross the street because it was too tired.”**

A single attention head might struggle to decide what **“it”** refers to. Multi-Head Attention solves this by letting different heads focus on different relationships simultaneously:

- **Head 1:** Connects **“it”** to **“animal”** (identifying the noun coreference).
- **Head 2:** Connects **“it”** to **“tired”** (identifying the state/property).
- **Head 3:** Connects **“cross”** to **“street”** (identifying verb-object structure).

By combining all heads, the model builds a rich, multi-dimensional understanding of the token **“it”**.

Causal Masking (Lower Triangular Mask)

In auto-regressive models (like GPT), predicting the next token requires that a position **cannot look ahead** to future tokens during training. **Causal Masking** enforces this by blocking attention scores between current tokens and future positions.

Mathematical Definition

Before applying Softmax, a mask matrix M is added to the scaled query-key scores:

$$\text{CausalAttention}(Q, K, V) = \text{softmax} \left(\left(\frac{QK^T}{\sqrt{d_k}} \right) + M \right) V$$

Where the mask matrix $M \in \mathbb{R}^{T \times T}$ is defined as:

$$M_{i,j} = \begin{cases} 0 & \text{if } i \geq j \\ -\infty & \text{if } i < j \end{cases}$$

Step-by-Step Breakdown

1. **Compute Raw Scores:** Calculate the attention matrix $S = \frac{QK^T}{\sqrt{d_k}}$ of size $T \times T$.
2. **Apply Mask M :** Add $-\infty$ to all elements above the main diagonal (where column index $j >$ row index i).
3. **Apply Softmax:** Because $e^{-\infty} = 0$, the attention weights for all future positions become strictly 0.
4. **Weighted Output:** Output for position i becomes a weighted sum strictly over tokens at positions $j \leq i$.

Intuitive Example

For the sentence: “**The cat sat**” ($T = 3$)

Without masking, the score matrix allows every word to attend to every other word:

$S = [\text{The} \rightarrow \text{The}, \text{The} \rightarrow \text{cat}, \text{The} \rightarrow \text{sat}; \text{cat} \rightarrow \text{The}, \text{cat} \rightarrow \text{cat}, \text{cat} \rightarrow \text{sat}; \text{sat} \rightarrow \text{The}, \text{sat} \rightarrow \text{cat}, \text{sat} \rightarrow \text{sat}]$

Adding the causal mask matrix M :

$$M = [0, -\infty, -\infty; 0, 0, -\infty; 0, 0, 0]$$

After applying Softmax, the resulting attention weight matrix becomes:

$$\text{Weights} = [1.0, 0.0, 0.0; 0.3, 0.7, 0.0; 0.2, 0.7, 0.1]$$

- Row 1 (“**The**”): Can only look at “**The**” (100%). Future words “**cat**” and “**sat**” are masked out to 0%.
- Row 2 (“**cat**”): Can look at “**The**” (30%) and “**cat**” (70%). Future word “**sat**” is masked out to 0%.
- Row 3 (“**sat**”): Can look at all previous words (“**The**”, “**cat**”, “**sat**”) because no future words remain.

Transformer Block: Layer Normalization & Residual Connections

1. Layer Normalization (LayerNorm)

What is Convergence?

Convergence means reaching a state where training stabilizes because the loss curve flattens out, indicating the model has minimized its error as much as it reasonably can.

Why Do We Need LayerNorm?

In deep Transformer networks, signal activations pass through dozens of stacked layers. Small variations at the input compound into massive spikes or vanishing signals in deeper layers (termed **internal covariate shift**). LayerNorm rescales activations across the feature dimension for each token independently, stabilizing signal flow and enabling faster, more stable convergence.

Mathematical Definition

For an input tensor $x \in \mathbb{R}^{T \times d_{\text{model}}}$, LayerNorm operates across the feature dimension d_{model} for each individual token vector $x_t \in \mathbb{R}^{d_{\text{model}}}$:

$$\text{LayerNorm}(x_t) = \gamma \odot \left(\frac{x_t - \mu}{\sqrt{\sigma^2 + \varepsilon}} \right) + \beta$$

Where:

- x_t : Input feature vector for a single token at position t of shape (d_{model}) .
- μ : Mean of the token's features, $\mu = \frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} x_{t,i}$ (Scalar).
- σ^2 : Variance of the token's features, $\sigma^2 = \frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} (x_{t,i} - \mu)^2$ (Scalar).
- ε : Tiny constant (e.g., 10^{-5}) added to avoid division by zero when variance is tiny.
- $\frac{x_t - \mu}{\sqrt{\sigma^2 + \varepsilon}}$: **Standardization step**, it forces features to have zero mean ($\mu = 0$) and unit variance ($\sigma^2 = 1$).
- γ : Learnable **scaling parameter** of shape (d_{model}) . Restores expressive power if pure unit variance is sub-optimal.
- β : Learnable **shift parameter** (bias) of shape (d_{model}) . Restores model expressivity by allowing shifted centers.

Key Difference: LayerNorm vs. BatchNorm

Metric	Batch Normalization (BatchNorm)	Layer Normalization (LayerNorm)
Normalization Axis	Across the batch dimension (N) for each feature.	Across the feature dimension (d_{model}) for each token.
Batch Dependency	High. Small or variable batch sizes break stability.	None. Works identically for batch size = 1 or variable lengths.
Primary Use Case	Computer Vision / Convolutional Networks.	Sequence Models / Transformers / LLMs.

2. Residual Connections (Skip Connections)

Why Do We Need Residual Connections?

In deep neural networks, passing data sequentially through dozens of layers (multiplications by weights + non-linear activations like GELU) degrades the signal. Information gets lost, and during

backpropagation, the training signals (gradients) vanish as they are repeatedly multiplied by small weight matrices.

Residual connections bypass a sub-layer by adding its original input directly to its output ($y = x + F(x)$). This creates an “information highway” that preserves both raw features and gradient signals.

Standard Transformer Block Architecture (Pre-LN)

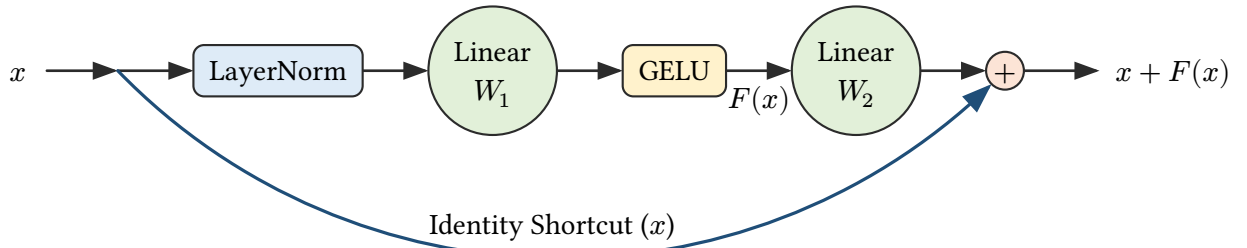
Modern LLMs (like LLaMA and GPT-3) use **Pre-Layer Normalization**, where LayerNorm is applied *before* the sub-layer function inside the residual connection:

1. Multi-Head Attention Sub-Layer:

$$x_{\text{attn}} = x + \text{MultiHeadAttention}(\text{LayerNorm}(x))$$

2. Feed-Forward Network (FFN) Sub-Layer:

$$x_{\text{out}} = x_{\text{attn}} + \text{FFN}(\text{LayerNorm}(x_{\text{attn}}))$$



Pre-LN Residual Connection with GELU Feed-Forward Sub-Layer

Mathematical Proof: Solving Vanishing Gradients

Consider a deep network with L residual layers. The output at final layer L is a direct recursive sum of earlier inputs and sub-layer transformations f_i :

$$x_L = x_0 + \sum_{i=0}^{L-1} f_i(x_i)$$

When computing the gradient of Loss ($\mathcal{L}$) with respect to early activations x_0 , we apply the chain rule:

$$\frac{\partial \mathcal{L}}{\partial x_0} = \frac{\partial \mathcal{L}}{\partial x_L} \cdot \frac{\partial x_L}{\partial x_0} = \frac{\partial \mathcal{L}}{\partial x_L} \cdot \left(1 + \frac{\partial}{\partial x_0} \sum_{i=0}^{L-1} f_i(x_i) \right)$$

Key Insight: Notice the “1” inside the bracket. Even if the transformed gradient term $\frac{\partial}{\partial x_0} \sum f_i(x_i)$ approaches zero (vanishes), the gradient simplifies to:

$$\frac{\partial \mathcal{L}}{\partial x_0} \approx \frac{\partial \mathcal{L}}{\partial x_L} \cdot (1)$$

This guarantees an unattenuated, direct path for training gradients to flow from the very last layer back to the input parameters without decaying to zero!

Transformer Blocks: Encoder vs. Decoder

Transformer Encoder Block

An Encoder block processes the entire input sequence simultaneously with bi-directional self-attention. It uses a **Pre-LayerNorm** architecture with two main sub-layers: Multi-Head Self-Attention (MHSA) and a Position-wise Feed-Forward Network (FFN).

Forward Pass Mathematics

Given input sequence embeddings $x_0 \in \mathbb{R}^{T \times d_{\text{model}}}$:

1. Sub-Layer 1: Multi-Head Self-Attention

$$x_1 = x_0 + \text{MultiHeadAttention}(\text{LayerNorm}(x_0))$$

2. Sub-Layer 2: Position-wise Feed-Forward Network

$$x_2 = x_1 + \text{FFN}(\text{LayerNorm}(x_1))$$

Output: Encoded representation matrix $x_2 \in \mathbb{R}^{T \times d_{\text{model}}}$.

Transformer Decoder Block

A full Encoder-Decoder style Transformer Decoder block consists of **three** sub-layers:

1. **Masked Multi-Head Self-Attention** (prevents attending to future tokens).
2. **Encoder-Decoder Cross-Attention** (attends to encoder outputs).
3. **Position-wise Feed-Forward Network (FFN)**.

Forward Pass Mathematics

Given target sequence input $y_0 \in \mathbb{R}^{T_{\text{tgt}} \times d_{\text{model}}}$ and Encoder Output $H_{\text{enc}} \in \mathbb{R}^{T_{\text{src}} \times d_{\text{model}}}$:

1. Sub-Layer 1: Masked Self-Attention (Causal)

$$y_1 = y_0 + \text{MultiHeadAttention}(\text{LayerNorm}(y_0), \text{mask} = \text{Causal})$$

Note: The causal mask sets future attention weights to $-\infty$ so position t cannot attend to tokens at $t + 1, t + 2, \dots$

2. Sub-Layer 2: Encoder-Decoder Cross-Attention

$$y_2 = y_1 + \text{CrossAttention}(Q = \text{LayerNorm}(y_1), K = H_{\text{enc}}, V = H_{\text{enc}})$$

Note: Query (Q) comes from the decoder state (y_1), while Key (K) and Value (V) come from the Encoder Output (H_{enc}).

3. Sub-Layer 3: Position-wise Feed-Forward Network

$$y_3 = y_2 + \text{FFN}(\text{LayerNorm}(y_2))$$

Output: Decoded representation matrix $y_3 \in \mathbb{R}^{T_{\text{tgt}} \times d_{\text{model}}}$.

- *LLMs have decoder only architecture.*

3. Architectural Comparison

Block Type	Attention Types Used	Causal Masking?	Example Models
Encoder Only	Bi-directional Self-Attention	No	BERT, RoBERTa, ViT

Decoder Only	Masked Causal Self-Attention	Yes	GPT-4, LLaMA, Claude
Encoder-Decoder	Causal Self-Attention + Cross-Attention	Yes (Self-Attn only)	T5, BART, Whisper

Complete Model Architecture and Training Loop

Modern LLMs like GPT use a decoder-only architecture. It consists of:

- Stack of decoder blocks:
 - Input: token embedding + positional encoding
 - Masked self-attention (causal mask)
 - Feed-Forward Network (FFN)
 - Output: Probability distribution over vocabulary
- Causal masked self-attention only (no cross-attention)
- Simpler, more efficient training for autoregressive language modeling

Mathematical pipeline:

$$\begin{aligned}x_{\text{embed}} &= \text{Embed}(x) + \text{PE} \\h^0 &= x_{\text{embed}} \\h^l &= \text{DecoderBlock}^l(h^{l-1}) \quad \text{for } l = 1, \dots, N \\ \text{logits} &= h^N \cdot W_{\text{output}} \\ \text{probs} &= \text{softmax}(\text{logits})\end{aligned}$$

Cross-Entropy Loss

Cross-entropy loss measures how far the model's predicted probability distribution deviates from the actual ground-truth next token.

$$\mathcal{L} = -\frac{1}{BT} \sum_{b=1}^B \sum_{t=1}^T \log(p_b(y_{b,t} \mid y_{b,1}, \dots, y_{b,t-1}))$$

where $\mathcal{L}$ is the average cross-entropy loss, $p_b(y_{b,t} \mid \dots)$ is the predicted probability assigned to the target token $y_{b,t}$ at sequence position t in batch sample b , B is the batch size, and T is the sequence length.

In terms of raw logits z , the loss for a single token at position (b, t) with true class index $k = y_{b,t}$ is:

$$\mathcal{L}_{b,t} = -z_{b,t,k} + \log\left(\sum_{j=1}^{|V|} \exp(z_{b,t,j})\right)$$

where $|V|$ is the vocabulary size and $z_{b,t,j}$ is the logit for token j .

Directly evaluating the exponential terms causes numerical overflow for large positive logits or underflow for large negative logits. To prevent this, frameworks use the **LogSumExp trick** by subtracting $m = \max_j(z_j)$:

$$\text{LSE}(z) = m + \log\left(\sum_{j=1}^{|V|} \exp(z_j - m)\right)$$

The numerically stable loss for a single token simplifies to:

$$\mathcal{L}_{b,t} = -z_{b,t,k} + \text{LSE}(z_{b,t})$$

Adam Optimizer

Instead of vanilla Stochastic Gradient Descent (SGD), which applies a single, fixed learning rate across all parameters regardless of their update frequency, the **Adam** (Adaptive Moment Estimation) optimizer dynamically adapts learning rates per parameter. It achieves this by combining two distinct mechanics:

1. **Momentum (1st Moment):** Keeps track of an exponentially decaying average of past gradients (m_t) to smooth out noise, maintain directional velocity, and accelerate convergence through flat regions.
2. **RMSProp (2nd Moment):** Keeps track of an exponentially decaying average of past **squared** gradients (v_t) to adaptively scale learning rates per parameter. It dampens step sizes for parameters with large, frequent gradients while boosting step sizes for parameters with small or sparse gradients.

Mathematical Formulation

For a given time step t , parameter vector θ_t , loss function $\mathcal{L}(\theta)$, and base learning rate η :

1. **Compute Gradient:**

$$g_t = \nabla_{\theta} \mathcal{L}(\theta_{t-1})$$

2. **Update Exponential Moving Averages (Biased Moments):**

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \end{aligned}$$

3. **Compute Bias Corrections:**

Since $m_0 = 0$ and $v_0 = 0$, both moments are heavily biased toward zero during initial steps. We correct them via:

$$\begin{aligned} \hat{m}_t &= \frac{m_t}{1 - \beta_1^t} \\ \hat{v}_t &= \frac{v_t}{1 - \beta_2^t} \end{aligned}$$

4. **Apply Parameter Update:**

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \varepsilon} \cdot \hat{m}_t$$

Where:

- θ_t : Vector of trainable model weights/parameters at step t .
- g_t : Gradient vector of the loss with respect to parameters at step t .
- m_t, v_t : Biased estimates of the 1st moment (mean gradient) and 2nd moment (uncentered variance of gradients), respectively.
- $\hat{m}_t, \hat{v}_t$: Unbiased (bias-corrected) 1st and 2nd moment estimates.
- η : Base learning rate hyperparameter (typically 10^{-3} to 10^{-4} for transformers).
- β_1 : Decay rate for the 1st moment estimate (standard default: 0.9).
- β_2 : Decay rate for the 2nd moment estimate (standard default: 0.999).
- β_1^t, β_2^t : Hyperparameters raised to the power of the current step number t (controlling the decay of bias correction over time).
- ε : Small constant added for numerical stability to prevent division by zero (standard default: 10^{-8}).

Data Pipeline and Training at Scale

Gradient Accumulation

When memory limits prevent fitting a large target batch size into GPU VRAM, **gradient accumulation** simulates a larger effective batch size. It computes gradients across N_{accum} smaller micro-batches and sums them up before calling the optimizer.

Algorithm

1. Accumulate Gradients over Micro-Batches:

$$g_{\text{accum}} = \sum_{k=1}^{N_{\text{accum}}} g_k$$

2. Average and Update Weights:

$$\theta_t = \text{OptimizerUpdate}\left(\theta_{t-1}, \frac{1}{N_{\text{accum}}} g_{\text{accum}}\right)$$

This yields an effective batch size of:

$$B_{\text{eff}} = B_{\text{micro}} \times N_{\text{accum}}$$

where B_{micro} is the per-device batch size per step, and g_k is the gradient computed on micro-batch k .

Learning Rate Scheduling

A learning rate schedule dynamically adjusts the global learning rate η_t throughout training to improve numerical stability during early iterations and refine convergence near the end.

Warmup Phase

Gradually ramps up the learning rate linearly from near zero to η_{max} over T_{warm} steps. This prevents large, destabilizing gradient steps when parameter weights are randomly initialized or unaligned.

$$\eta_t = \eta_{\text{max}} \cdot \min\left(\frac{t}{T_{\text{warm}}}, 1\right) \quad \text{for } 1 \leq t \leq T_{\text{warm}}$$

Cosine Decay Phase

After warmup, reduces the learning rate to a minimum baseline η_{min} along a smooth cosine curve over the remaining steps T_{max} . This allows the optimizer to make fine-grained parameter adjustments as it approaches a local minimum.

$$\eta_t = \eta_{\text{min}} + \frac{1}{2}(\eta_{\text{max}} - \eta_{\text{min}}) \cdot \left(1 + \cos\left(\pi \frac{t - T_{\text{warm}}}{T_{\text{max}} - T_{\text{warm}}}\right)\right) \quad \text{for } t > T_{\text{warm}}$$

Symbol Definitions

- η_t : Global learning rate at step t .
- η_{max} : Peak target learning rate after warmup.
- η_{min} : Minimum baseline learning rate (often $0.1 \cdot \eta_{\text{max}}$ or 0).
- T_{warm} : Total number of warmup steps.
- T_{max} : Total number of training steps.

Evaluation Metrics

Perplexity (PPL)

- Perplexity is the standard intrinsic metric for language models. It quantifies model uncertainty by measuring how “surprised” the model is when predicting the next token across a test corpus of length N .
- Mathematically, perplexity is the exponential of the average cross-entropy loss:

$$\text{PPL} = \exp(\mathcal{L}) = \exp\left(-\frac{1}{N} \sum_{i=1}^N \log p(y_i \mid y_1, \dots, y_{i-1})\right)$$

Interpretation: **A perplexity of K means the model is, on average, as uncertain as if it were choosing uniformly at random among K equiprobable candidate tokens. Lower is Better:** Lower perplexity indicates higher probability assigned to the ground-truth text sequence.